F.CSM302 — Үйлдлийн систем

Бие даалт 1

Судалгаа, хэрэгжилт, туршилтын тайлан

Fork-Join Sorting Application

Сурах бичгийн Бүлэг 4 — Programming Project 3
UI дүрслэл: **Recursion tree**

Хуваарилалтын код: **IV-2-7**

Багийн гишүүд

Оюутны код	Нэр
B200910032	Д.Дагиймаа
B221940034	Д.Оюунжаргал
B241940014	Г.Мягмарсүрэн
B241940127	О.Энх
B242270084	Э.Бүрэнбаяр

2026 оны 10-р сар

Гарчиг

Хураангуй **3**

1 Удиртгал **4**
1.1 Зорилго . . . 4
1.2 Ач холбогдол . . . 4
1.3 Хамрах хүрээ . . . 4

2 Сурах бичгийн шаардлага **4**

3 Онолын үндэс **5**
3.1 Fork-Join загвар . . . 5
3.2 Яагаад `RecursiveAction` вэ . . . 6
3.3 Threshold-ийн онол . . . 6
3.4 Mergesort ба Quicksort — зарчмын ялгаа . . . 6
3.5 Гурван талт хуваалт (Dutch national flag) . . . 6

4 Системийн зохиомж **7**
4.1 Компонентууд . . . 7
4.2 Өгөгдлийн урсгал . . . 7
4.3 Бүртгэлийн синхрончлол . . . 7

5 Хэрэгжилт **7**
5.1 Хэл, сан, орчин . . . 7
5.2 Mergesort — гол функц . . . 7
5.3 Quicksort — гол функц . . . 8
5.4 Pivot сонголт — илрүүлсэн алдаа ба засвар . . . 8
5.5 Стекийн хамгаалалт . . . 9
5.6 Рекурсийн модны байрлуулалт . . . 9

6 Туршилт ба UI дүрслэл **10**
6.1 Юуг өөрчилж турших вэ . . . 10
6.2 Дүрслэлээс юуг ажиглах вэ . . . 10

7 Туршилтын арга **11**
7.1 Орчин . . . 11
7.2 Туршилтын өгөгдөл . . . 11
7.3 Хүлээгдсэн үр дүн ба таамаглал . . . 12

8 Үр дүн **12**
8.1 Зөв ажиллагаа . . . 12
8.2 Туршилт A — threshold-ийн нөлөө . . . 12
8.3 Туршилт B — урсгалын тооны нөлөө . . . 13
8.4 Туршилт C — өгөгдлийн хэв маягийн нөлөө . . . 13
8.5 Pivot сонголтын засварын нөлөө . . . 13

9 Хэлэлцүүлэг **14**
9.1 Таамаглал бүрийн үнэлгээ 14
9.2 Үр дүнгийн тайлбар 14
9.3 Хязгаарлалт 14
9.4 Цаашид хийж болох ажил 15

10 Дүгнэлт **15**
10.1 Хэрэгжсэн зүйл 15
10.2 Нотолсон үр дүн 15
10.3 Сурсан зүйл 15

Хураангуй

Асуудал. Сурах бичгийн Бүлэг 4-ийн Programming Project 3 нь олон урсгалт эрэмбэлэх программыг Java-гийн *fork-join* параллель API-аар дахин хэрэгжүүлэхийг шаарддаг — Quicksort, Mergesort гэсэн хоёр хувилбараар, `RecursiveAction` ашиглан, тодорхой *threshold*-оос доош энгийн алгоритм руу шилжих байдлаар. Багшийн нэмэлт шаардлага нь рекурсив задралыг **рекурсийн мод** хэлбэрээр интерактив харуулах явдал юм.

Шийдэл. Java 21 дээр гадны сангүйгээр хэрэгжүүлэв. Хоёр алгоритм тус бүр `RecursiveAction`-ийг өргөтгөнө. `compute()` дуудлага бүр нь *thread-safe* бүртгэлд нэг зангилаа үлдээдэг бөгөөд Swing дээр бичсэн самбар эдгээрээс модыг барьж, зангилаа бүрийг *гүйцэтгэсэн ажилчин урсгалаар нь* өнгөөр ялгаж харуулна. Массивын хэмжээ, threshold, урсгалын тоо, өгөгдлийн хэв маяг, өөрийн оролт — бүгдийг UI-аас өөрчилж дахин турших боломжтой.

Туршилт. Нийт **58** автомат тест (номын жишээ, захын оролт, бүх өгөгдлийн хэв маяг, threshold 1–20 000, урсгал 1–8, 30 санамсаргүй ажиллуулалт, хүчингүй оролт, $n = 300\,000$) — бүгд давсан. Гүйцэтгэлийн хэмжилтийг гурван туршилтаар авсан: threshold-ийн нөлөө, урсгалын тооны нөлөө, өгөгдлийн хэв маягийн нөлөө.

Гол үр дүн.

1. Хоёр цөмт машин дээр threshold ≥ 50 үед хурдасгалт $\approx \mathbf{2.0}\times$ — онолын дээд хязгаарт хүрсэн.
2. Threshold $= 1$ үед хурдасгалт $\mathbf{0.8}\times$ болж буурна: fork-ийн зардал ашигтай ажлаас давамгайлав. Threshold-ийн оновчтой муж нь **50–500**.
3. Quicksort дээр *гурван талт* хуваалт хэрэглэснээр давхардсан утгатай өгөгдөл дээр мод 16 383 зангилаанаас **1** зангилаа болж багасав.
4. Pivot-ыг тогтмол байрлалаар сонговол эрэмбэлэгдсэн өгөгдөл дээр рекурсийн гүн $O(n)$ болж *стек халдгийг* илрүүлж, хэш дээр суурилсан сонголтоор зассан.

1 Удиртгал

1.1 Зорилго

Энэ ажлын зорилго нь Java-гийн fork-join параллелизмыг ашиглан *хувааба эзэгнэ* (divide-and-conquer) төрлийн эрэмбэлэх алгоритмуудыг хэрэгжүүлж, параллель ажлын задралыг *нүдэнд харагдахаар* болгох явдал юм.

1.2 Ач холбогдол

Fork-join загвар нь зөвхөн эрэмбэлэлтийн асуудал биш. Орчин үеийн олон цөмт процессор дээр ажлыг рекурсив задлан, ажилчин урсгалууд хоорондоо ажил *хулгайлж* (work stealing) ачааллыг тэнцвэржүүлэх энэ загвар нь Java-гийн `parallelStream`, `Arrays.parallelSort`, мөн C++, .NET, Rust-ын параллель сангуудын үндэс болдог. Сурах бичгийн 4.5.2 хэсэгт үүнийг *далд урсгалчлал* (implicit threading) гэж үздэг — программист урсгалыг өөрөө үүсгэдэггүй, зөвхөн *ажлыг* задалдаг.

Мөн энэ төсөл нь 4-р бүлгээс гадна 6–7-р бүлгийн синхрончлолын асуудлыг практикт хөндөг: модны бүртгэлийг олон урсгал зэрэг бичдэг тул *thread-safe* байх ёстой.

1.3 Хамрах хүрээ

Хамрагдсан	**Хамрагдаагүй**
Mergesort, Quicksort — fork-join хувилбар	Бусад эрэмбэлэх алгоритм (heapsort, radix)
`RecursiveAction`, `ForkJoinPool`	Бусад параллель загвар (Executor, виртуал урсгал)
Рекурсийн модны интерактив дүрслэл	Тархсан (олон машин) хувилбар
Threshold, урсгал, өгөгдлийн хэв маягийн туршилт	Дискэн дээрх (external) эрэмбэлэлт
`Comparable` интерфейс дэмжсэн ерөнхий төрөл	Тогтвортой (stable) эрэмбэлэлтийн баталгаа

2 Сурах бичгийн шаардлага

Programming Project 3 (Fork-Join Sorting Application) нь Project 2 (Multithreaded Sorting Application)-ийн үргэлжлэл бөгөөд дараах зүйлсийг шаарддаг:

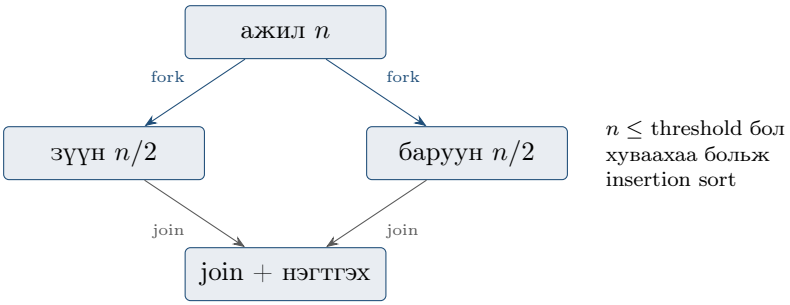
#	Номын шаардлага	Хуудас	Энэ ажилд хэрхэн хангасан
1	Project 2-ыг Java-гийн fork-join API-аар дахин хэрэгжүүлэх	P-27	`MergesortAction`, `QuicksortAction` нь `ForkJoinPool` дотор ажиллана
2	*Хоёр* хувилбар: Quicksort ба Mergesort	P-27	UI-аас сонгоно; хоёулаа бүрэн хэрэгжсэн
3	Threshold-оос доош энгийн алгоритм (Selection эсвэл Insertion) руу шилжих — ном “size 100 or fewer” гэж жишээ татсан	P-28	`SequentialSort.insertionSort()`; командын мөрийн өгөгдмөл threshold яг **100**
4	`RecursiveTask` биш `RecursiveAction` өргөтгөх (эрэмбэлэлт утга буцаахгүй), Figure 4.19-ийг үз	P-28 Fig. 4.19: х. 184	Хоёр класс хоёулаа `RecursiveAction`
5	Объектууд Java-гийн `Comparable` интерфейсийг хэрэгжүүлсэн байх	P-28	`<T extends Comparable<? super T>>` ерөнхий параметр
6	*Багшийн нэмэлт:* рекурсийн модыг UI-д харуулах	—	`TreePanel` — fork-join задрал, дэд ажлууд, join үе шат
7	*Багшийн нэмэлт:* өгөгдлийн хэмжээ ба threshold-ийг өөрчилж турших	—	UI-ийн зүүн самбар + командын мөрийн параметрүүд

3 Онолын үндэс

3.1 Fork-Join загвар

Эх сурвалж: §4.5.2 Fork Join (х. 180), §4.5.2.1 Fork Join in Java (х. 180–183), Figure 4.16 (х. 181), Figure 4.17 (х. 182), Figure 4.18 — `SumTask` код (х. 183), Figure 4.19 — UML (х. 184).

Fork-join нь *хуваа ба эзэгнэ* зарчмыг параллель болгосон хэлбэр:



- `fork()` — дэд ажлыг ээлжинд тавина. *Шинэ урсгал үүсгэдэггүй.*
- `join()` — тэр дэд ажил дуустал хүлээнэ.
- `invokeAll(a, b)` — `a.fork(); b.compute(); a.join();` -тэй дүйцнэ. Нэг ажлыг одоогийн урсгал *өөрөө* гүйцэтгэдэг тул дэмий fork хийхгүй.

Work stealing. `ForkJoinPool` дотор цөмийн тоотой тэнцүү ажилчин урсгал байна. Урсгал бүр өөрийн *deque*-тэй: өөрийн ажлыг толгойноос нь авч, ажилгүй болсон урсгал өөр урсгалын дараалллын *сүүлээс* ажил хулгайлна. Иймээс сая ширхэг `fork()` хийсэн ч ердөө хэдэн урсгал ажиллана. Манай дүрслэлд энэ нь *өнгөний холилдол* болж шууд харагдана.

3.2 Яагаад `RecursiveAction` вэ

	`RecursiveTask<V>`	`RecursiveAction`
`compute()`	`V` буцаана	`void`
Хэзээ	Нийлбэр, тоолол — *үр дүн* буцаах үед	**Эрэмбэлэлт** — массивыг *газар дээр нь* өөрчилнө
Номын жишээ	`SumTask` (4.5.2.1)	Энэ төсөл

3.3 Threshold-ийн онол

Threshold k-ийн сонголт нь хоёр зардлын тэнцвэр:

$$T(n) \;\approx\; \underbrace{\frac{n}{k}\cdot c_{\text{fork}}}_{\text{задралын зардал}} \;+\; \underbrace{\frac{n}{k}\cdot O(k^2)}_{\text{навч дээрх insertion sort}} \;+\; \underbrace{O\!\left(n\log\frac{n}{k}\right)}_{\text{нэгтгэх}}$$

Эхний гишүүн k *өсөхөд буурна*, хоёр дахь нь k өсөхөд *шугаман өснө* $(n\cdot k)$. Иймээс дунд нь оновчтой цэг байдаг. Сурах бичиг “жишээ нь 100 буюу түүнээс бага” гэж санал болгодог — бидний хэмжилт үүнийг баталсан.

3.4 Mergesort ба Quicksort — зарчмын ялгаа

	Mergesort	**Quicksort**
Гол ажил хаана	Хуваалтын *дараа* (merge, $O(n)$)	Хуваалтын *өмнө* (partition, $O(n)$)
Хуваалт	Үргэлж яг тэнцүү	Pivot-оос хамаарна
Модны хэлбэр	Төгс тэнцвэртэй, гүн $=\log_2(n/k)$	Тэгш бус, гүн нь pivot-ийн чанараас хамаарна
Нэмэлт санах ой	$O(n)$ туслах массив	$O(\log n)$ стек
Параллельчлалын саад	*merge* цуваа байна	Бараг байхгүй
Хамгийн муу тохиолдол	$O(n\log n)$ баталгаатай	$O(n^2)$ (pivot муу үед)

3.5 Гурван талт хуваалт (Dutch national flag)

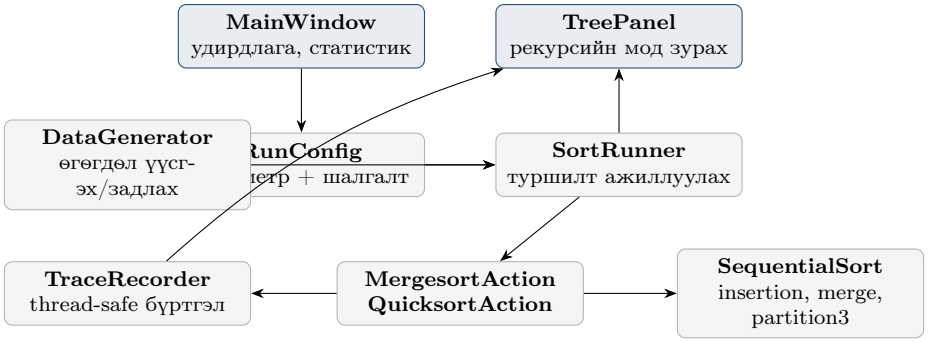
Энгийн хоёр талт хуваалт нь *давхардсан утгыг* зохицуулж чаддаггүй. Бүх элемент ижил бол нэг тал нь хоосон, нөгөө нь $n-1$ болж, рекурсийн гүн $O(n)$ — стек халина. Dijkstra-гийн гурван талт хуваалт мужийг *гурав* хуваана:

$$\underbrace{a[lo..lt)}_{<v} \qquad \underbrace{a[lt..gt]}_{=v\ \text{(эцсийн байрандаа)}} \qquad \underbrace{a(gt..hi)}_{>v}$$

Тэнцүү хэсгийг дахин хөндөхгүй тул бүх элемент ижил үед *нэг* хуваалтаар ажил дуусна.

4 Системийн зохиомж

4.1 Компонентууд



Гол зохиомжийн зарчим: *алгоритмын давхарга нь UI-ээс бүрэн хамааралгүй.* `SortRunner` нь `RunConfig` авч `RunResult` буцаана; Swing-ийн ямар ч класс импортлодоггүй. Иймээс `-headless`, `-bench`, тестийн багц гурвуулаа дэлгэцгүйгээр ажиллана.

4.2 Өгөгдлийн урсгал

1. Хэрэглэгч UI-аас параметр өгнө → `RunConfig`.
2. `RunConfig.validate()` — хүчингүй утгыг *энд* барьж, ойлгомжтой мессеж шиднэ.
3. `DataGenerator` массив үүсгэнэ (эсвэл хэрэглэгчийн мөрийг задална).
4. **Жишиг хэмжилт**: ижил алгоритм, ижил threshold-оор *нэг урсгалаар* эрэмбэлж хугацааг тэмдэглэнэ.
5. **Fork-join хэмжилт**: `ForkJoinPool`-д даалгавар өгнө. `compute()` дуудлага бүр `TraceRecorder`-т зангилаа нэмнэ.
6. **Шалгалт**: `Arrays.sort`-ын үр дүнтэй тулгана.
7. `RunResult` → UI: `TreePanel` мод зурна, статистик самбарт тоонууд гарна.

4.3 Бүртгэлийн синхрончлол

`compute()` нь олон урсгалаас *зэрэг* дуудагддаг тул бүртгэл thread-safe байх ёстой. Цоож ашиглавал хэмжилтийг гажуудуулах тул цоожгүй бүтэц сонгосон:

`AtomicInteger`	Зангилааны дугаарыг атомикаар олгоно (`getAndIncrement`) — цоожгүй
`ConcurrentLinkedQueue`	Зангилааг нэмэх нь lock-free
`volatile` талбарууд	Нэг урсгал бичээд өөр урсгал (UI) уншина — харагдах байдлыг баталгаажуулна

5 Хэрэгжилт

5.1 Хэл, сан, орчин

Java 21 (хэл: Java 17 түвшний боломжууд). **Гадны сан ашиглаагүй** — `java.util.concurrent` ба Swing нь JDK-ийн бүрэлдэхүүн. Build хэрэгсэл шаардлагагүй: `javac` хангалттай.

5.2 Mergesort — гол функц

```
@Override
protected void compute() {
```

```
    TraceNode node = tr.open(parentId, lo, hi, depth, a);
    int myId = (node == null) ? -1 : node.id;

    if (hi - lo <= threshold) {                 // ===== НАВЧ =====
        SequentialSort.insertionSort(a, lo, hi);
        tr.close(node, true, a);
        return;
    }

    int mid = lo + (hi - lo) / 2;               // үргэлж ТЭНЦҮҮ хуваагдана
    invokeAll(                                   // = fork + compute + join
        new MergesortAction<>(a, tmp, lo,  mid, threshold, tr, myId, depth + 1),
        new MergesortAction<>(a, tmp, mid, hi,  threshold, tr, myId, depth + 1));

    SequentialSort.merge(a, tmp, lo, mid, hi);  // нэгтгэх нь ПАРАЛЛЕЛЬЧЛАГДАХГҮЙ
    tr.close(node, false, a);
}
```

`merge` дотор нэг оновчлол бий: `if (a[mid-1] <= a[mid]) return;` — хоёр хэсэг аль хэдийн зөв дараалалтай бол хуулалт хийхгүй. Эрэмбэлэгдсэн өгөгдөл дээр энэ нь ажиллах хугацааг 153 мс-ээс **6.8 мс** болгон бууруулсан (Хүснэгт 8.4).

5.3 Quicksort — гол функц

```
int[] r = SequentialSort.partition3(a, lo, hi);   // ЦУВАА ажил
int ll = lo,     lr = r[0];                        // pivot-оос БАГА хэсэг
int rl = r[1]+1, rr = hi;                          // pivot-оос ИХ хэсэг
// r[0]..r[1] нь pivot-тай тэнцүү -- эцсийн байрандаа, дахин хөндөхгүй

boolean hasL = lr - ll > 0, hasR = rr - rl > 0;
if (hasL && hasR) {
    invokeAll(new QuicksortAction<>(a, ll, lr, threshold, tr, myId, depth + 1),
              new QuicksortAction<>(a, rl, rr, threshold, tr, myId, depth + 1));
} else if (hasL) {
    new QuicksortAction<>(a, ll, lr, threshold, tr, myId, depth + 1).compute();
} else if (hasR) {
    new QuicksortAction<>(a, rl, rr, threshold, tr, myId, depth + 1).compute();
}
```

Хоосон тал үүсвэл даалгавар *үүсгэхгүй* — мод цэвэрхэн болж, дэмий fork хийхгүй.

5.4 Pivot сонголт — илрүүлсэн алдаа ба засвар

Эхний хувилбарт медиан-3-ыг (эхний, дунд, сүүлчийн элемент) ашигласан бөгөөд медианыг `lo` байрлалд *урьдчилан зөөдөг* байв. Туршилтаар эрэмбэлэгдсэн өгөгдөл дээр Quicksort **2310 мс** зарцуулж, модны гүн хамгаалалтын хязгаар (48)-д хүрч байлаа.

Шалтгаан. Урьдчилсан зөөлт нь хамгийн бага элементийг хэсгийн *төгсгөлд* үлдээдэг. Дараагийн түвшинд медиан-3 нь (бага, дунд, *хамгийн бага*) гурвын медианыг — өөрөөр хэлбэл бараг хамгийн бага утгыг — сонгоно. Хуваалт $1 : n-2$ болж, *энэ хэв маяг түвшин бүрт давтагдана.* Үр дүнд нь гүн $O(n)$.

Засвар. Pivot-ийн *байрлалуудыг* (lo, hi)-ээс гаргасан SplitMix хэш-ээр сонгоно:

```
private static int pick(int lo, int hi, int k) {
    long h = lo * 0x9E3779B97F4A7C15L ^ hi * 0xBF58476D1CE4E5B9L
           ^ k * 0x94D049BB133111EBL;
    h ^= (h >>> 31); h *= 0x7FB5D329728EA185L; h ^= (h >>> 29);
    return lo + (int) Math.floorMod(h, (long) (hi - lo));
}
```

Хэш нь өгөгдлийн агуулгатай ямар ч хамааралгүй тул дээрх давхарлалыг тасална. Мөн зөвхөн (lo, hi)-ээс хамаардаг тул үр дүн **давтагдана** — ижил тохиргоо ижил мод өгнө.

Үр дүн: эрэмбэлэгдсэн өгөгдөл дээр 2310 мс → **196 мс**, гүн 48 → 27.

5.5 Стекийн хамгаалалт

Квиксортын гүн онолын хувьд $O(n)$ болох магадлал үлддэг тул `MAX_PARALLEL_DEPTH = 48` хязгаар тавьсан: түүнээс цааш `quicksort3()` гэсэн *давталттай* (tail-call eliminated) нэг урсгалт хувилбар руу шилжинэ. Тэр нь бага талыг рекурсээр, их талыг давталтаар боловсруулдаг тул стекийн гүн $O(\log n)$-оос хэтрэхгүй.

5.6 Рекурсийн модны байрлуулалт

Зангилааны дугаар нь *үүсэх дарааллаар* олгогддог бөгөөд олон урсгал зэрэг ажилладаг тул **дугаарын дараалал нь модны бүтцийг илэрхийлэхгүй**. Иймээс эцгийн дугаараар хүүхдүүдийг бүлэглэж, `lo`-оор эрэмбэлж, гүнзгийрүүлсэн хайлтаар (DFS) байрлалыг тооцоолно:

- навч бүр дараалсан x байрлал эзэлнэ;
- дотоод зангилааны x нь *хүүхдүүдийнхээ дундаж*;
- y нь рекурсийн гүн.

Ингэснээр ирмэгүүд огтлолцохгүй, мод уншигдахуйц болно.

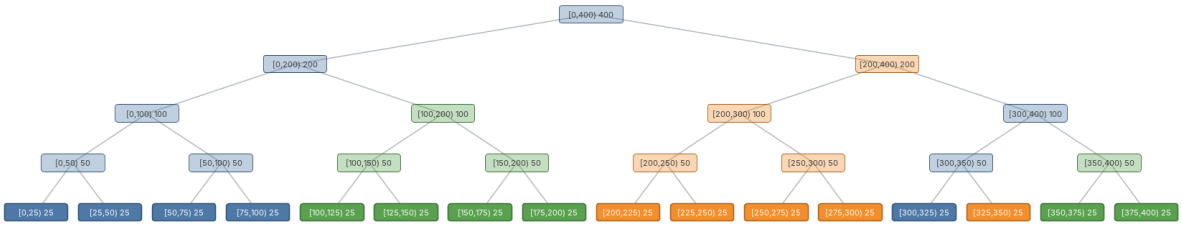
6 Туршилт ба UI дүрслэл

6.1 Юуг өөрчилж турших вэ

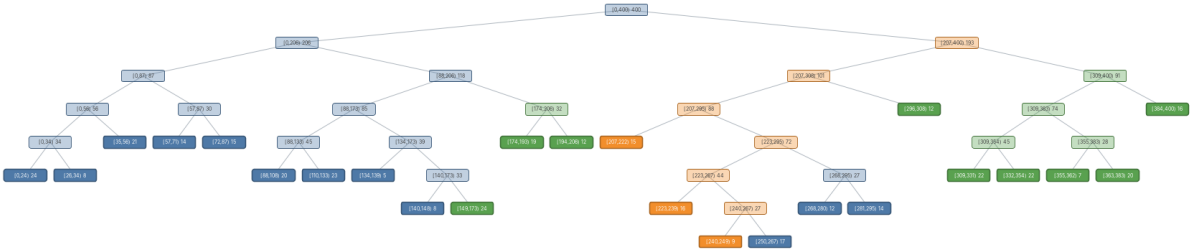
Өөрчлөх зүйл	Модонд юу өөрчлөгдөх	Юуг нотлох
Threshold	Модны *гүн* ба навчны тоо	Threshold нь рекурсийн гүнийг шууд тодорхойлно; хэт жижиг бол fork-ийн зардал давамгайлна
Алгоритм	Модны *хэлбэр*	Mergesort тэгш хэмтэй, Quicksort pivot-оос хамаарч тэгш бус
Өгөгдлийн хэв маяг	Quicksort-ийн модны тэнцвэр	Гурван талт хуваалтын ач тус (ALL_EQUAL дээр ганц зангилаа)
Урсгалын тоо	Өнгөний тоо ба тархалт	Work stealing — ажил урсгалуудын хооронд динамикаар хуваарилагдана
Массивын хэмжээ	Модны *өргөн*	n хоёр дахин өсөхөд мод нэг түвшин өндөрснө
Seed	Өгөгдөл өөрчлөгдөнө	Үр дүн давтагдах эсэх

6.2 Дүрслэлээс юуг ажиглах вэ

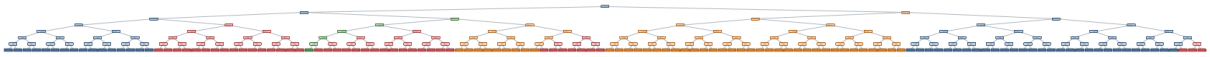
1. **Зангилаа = `compute()` дуудлага.** Шошго нь тухайн дуудлагын хариуцсан `[lo,hi)` муж.
2. **Өнгө = ажилчин урсгал.** Нэг дэд мод олон өнгөтэй байвал — тэр ажил *хулгайлагдсан* гэсэн үг.
3. **Дүүрэн өнгө = навч.** Яг энд рекурс зогсож insertion sort ажилласан. Навчны тоо $\approx n/\text{threshold}$.
4. **Гүн.** Mergesort-д яг $\lceil \log_2(n/k) \rceil$; Quicksort-д илүү их бөгөөд өгөгдлөөс хамаарна.
5. **Алхамчилсан горим** нь зангилааг *үүссэн дарааллаар* нээдэг тул fork-ийн тархалтыг хугацааны дарааллаар харж болно.



Зураг 1: Mergesort, $n = 400$, threshold $= 25$, 4 урсгал. Мод *төгс тэнцвэртэй*: түвшин бүрт хэмжээ яг хоёр дахин багасна. Өнгө нь ажлыг дөрвөн ажилчин урсгал хооронд хуваасныг харуулна.



Зураг 2: Quicksort, ижил өгөгдөл ба тохиргоо. Хуваалт *тэгш бус* — үндэс $[0, 400)$ нь 206 ба 193 болж хуваагдав (нэг элемент нь pivot). Зарим зангилаа ганц хүүхэдтэй: нөгөө тал нь хоосон болсон.



Зураг 3: Ижил өгөгдөл ($n = 400$, Mergesort) дээр threshold-ийг 25 → 5 болгов. Мод хоёр түвшин өндөрсөж, навчны тоо дөрөв дахин нэмэгдэв — threshold ба рекурсийн гүний хамаарлын шууд нотолгоо.

7 Туршилтын арга

7.1 Орчин

Үзүүлэлт	Утга
Java	OpenJDK **21.0.12.1**
OS	Linux x86_64
Процессор	**2 логик цөм**
Хэмжилтийн горим	`-bench`, модны бүртгэлгүй (`-no-trace`)
JIT халаалт	Хэмжилт бүрийн өмнө 200 000 элементээр нэг удаа халаана
Жишиг (baseline)	*Ижил* алгоритм, *ижил* threshold, нэг урсгалаар

Жишгийг ижил алгоритм, ижил threshold-оор авсан нь чухал: ингэснээр хурдасгалтын тоо нь *зөвхөн параллельчлалын* нөлөөг хэмжинэ, алгоритмын ялгаа хольцгүй болно.

7.2 Туршилтын өгөгдөл

Таван хэв маяг: `RANDOM`, `SORTED`, `REVERSED`, `FEW_UNIQUE` (5 ялгаатай утга), `ALL_EQUAL`. Бүгд тогтмол seed-тэй тул давтагдана.



Зураг 4: Quicksort, бүх элемент ижил ($n = 400$). Гурван талт хуваалтын ачаар мод **ганц зангилаатай** — нэг хуваалтаар бүх элемент эцсийн байрандаа суув. Энгийн хоёр талт хуваалт бол энд 400 түвшинтэй мод үүсгэж стек халах байв.

7.3 Хүлээгдсэн үр дүн ба таамаглал

#	Таамаглал	Үндэслэл
H1	Threshold хэт жижиг (1–10) үед хурдасгалт < 1 болно	Fork-ийн зардал ашигтай ажлаас давамгайлна
H2	Threshold 50–500 орчимд хурдасгалт хамгийн өндөр	Задралын ба insertion sort-ийн зардлын тэнцвэр
H3	Хурдасгалт цөмийн тооноос хэтрэхгүй	Amdahl-ын хууль; merge нь цуваа
H4	Mergesort-ийн модны гүн өгөгдлөөс хамаарахгүй, Quicksort-ийнх хамаарна	Mergesort үргэлж яг дундуур хуваана
H5	`ALL_EQUAL` дээр Quicksort хамгийн хурдан	Гурван талт хуваалт бүх элементийг нэг алхамд эцсийн байранд нь суулгана

8 Үр дүн

8.1 Зөв ажиллагаа

```
$ ./test.sh
=== ДҮН: 58 давсан, 0 унасан ===
```

#	Тестийн бүлэг	Тоо	Хамрах хүрээ
1	Номын жишээ (Figure 4.27)	4	Хоёр алгоритм, үр дүн ба мод
2	Захын оролт	15	$n = 0, 1, 2, 3, 7, 8, 9$; threshold $> n$
3	Бүх өгөгдлийн хэв маяг	10	5 хэв маяг $\times$ 2 алгоритм
4	Threshold хувилбарууд	12	$1, 2, 10, 100, 5000, 20000$
5	Урсгалын тоо	8	$1, 2, 4, 8$
6	Санамсаргүй ажиллуулалт	1	30 ажиллуулалт, бүх параметр санамсаргүй
7	Хүчингүй оролт	6	threshold ≤ 0, урсгал $= 0$, $n < 0$, текст оролт
8	Том массив	2	$n = 300\,000$
	Нийт	**58**	бүгд давсан

Зөв эсэхийг бүх тохиолдолд `java.util.Arrays.sort`-ын үр дүнтэй элемент бүрээр нь тулган шалгасан.

8.2 Туршилт A — threshold-ийн нөлөө

$n = 1\,000\,000$, `RANDOM`, 2 урсгал.

	Mergesort			Quicksort		
Threshold	парал. мс	цуваа мс	хурдасгалт	парал. мс	цуваа мс	хурдасгалт
1	435.0	354.5	**0.81**	298.9	513.7	1.72
10	266.3	308.5	1.16	228.7	490.8	**2.15**
50	179.1	356.3	1.99	230.8	460.3	1.99
100	226.8	417.4	1.84	320.3	651.4	2.03
500	708.2	1367.6	1.93	577.2	1116.1	1.93
1000	1243.5	2546.8	2.05	999.3	1936.5	1.94
5000	4376.8	9021.0	2.06	4199.9	7797.0	1.86

Хамгийн бага *абсолют* хугацаа: Mergesort threshold = 50 үед **179 мс**, Quicksort threshold = 10 үед **229 мс**. Threshold $\geq$ 500-аас хойш абсолют хугацаа огцом өснө — навч дээрх insertion sort-ийн $O(k^2)$ зардал давамгайлж байна.

8.3 Туршилт B — урсгалын тооны нөлөө

$n = 1\,000\,000$, threshold = 100, `RANDOM`.

	Mergesort		Quicksort	
Урсгал	мс	хурдасгалт	мс	хурдасгалт
1	474.3	0.98	499.4	1.00
2	246.0	**1.95**	300.1	**2.03**
4	242.3	1.68	263.9	1.89

Нэг урсгалтай үед хурдасгалт ≈ 1.00 — хэмжилтийн арга зүй зөв болохын шалгалт. Хоёр урсгалтай үед ≈ 2.0 буюу *бараг шугаман*. Дөрвөн урсгал нь хоёр цөм дээр *нэмэр өгөхгүй* — бага зэрэг буурсан нь oversubscription-ийн зардал.

8.4 Туршилт C — өгөгдлийн хэв маягийн нөлөө

$n = 500\,000$, threshold = 100, 2 урсгал.

	Mergesort			Quicksort		
Хэв маяг	мс	гүн	зангилаа	мс	гүн	зангилаа
RANDOM	153.3	13	16383	213.6	27	16941
SORTED	**6.8**	13	16383	196.1	27	16903
REVERSED	101.9	13	16383	198.5	25	16773
FEW_UNIQUE	66.4	13	16383	18.5	3	5
ALL_EQUAL	6.7	13	16383	**1.7**	**0**	**1**

8.5 Pivot сонголтын засварын нөлөө

	Засварын өмнө		Засварын дараа	
Хэв маяг ($n = 200\,000$)	мс	гүн	мс	гүн
SORTED	726	48 (хязгаар)	**172**	15
REVERSED	574	48 (хязгаар)	**37**	15
RANDOM	274	16	177	17

9 Хэлэлцүүлэг

9.1 Таамаглал бүрийн үнэлгээ

#	Таамаглал	Үр дүн	Тайлбар
H1	Threshold жижиг үед хурдасгалт < 1	**Хэсэгчлэн**	Mergesort-д баталгаажсан (0.81). Quicksort-д threshold = 1 үед ч 1.72 байв — учир нь түүний цуваа жишиг нь `quicksort3` рекурс бөгөөд мөн адил зардалтай
H2	Оновчтой threshold 50–500	**Баталгаажсан**	Абсолют хугацааны минимум: Mergesort 50, Quicksort 10–50
H3	Хурдасгалт цөмийн тооноос хэтрэхгүй	**Баталгаажсан**	Дээд утга 2.15; 4 урсгал нэмэр өгөөгүй
H4	Mergesort-ийн гүн өгөгдлөөс хамаарахгүй	**Баталгаажсан**	Бүх хэв маягт яг **13** ($\log_2(500000/100) \approx 12.3$)
H5	`ALL_EQUAL` дээр Quicksort хамгийн хурдан	**Баталгаажсан**	1.7 мс, *ганц* зангилаа

9.2 Үр дүнгийн тайлбар

Threshold бол хамгийн чухал параметр. Threshold $1 \to 50$ болгоход Mergesort-ийн хугацаа 435 мс-ээс 179 мс болж *хоёр дахин* сайжирсан. Энэ нь алгоритм биш, зөвхөн *задралын нарийвчлалын* өөрчлөлт. Хэт жижиг threshold нь сая ширхэг жижиг даалгавар үүсгэж, тэдгээрийг ээлжинд оруулах, хулгайлах, join хийх зардал нь ашигтай ажлаас давамгайлна. Хэт том threshold нь эсрэгээрээ: insertion sort нь $O(k^2)$ тул threshold = 5000 үед нэг навч 25 сая үйлдэл хийнэ.

Amdahl-ын хууль практикт. Mergesort-ийн `merge` алхам нь параллельчлагдахгүй цуваа хэсэг. Хоёр цөм дээр 1.95 хурдасгалт гарсан нь цуваа хэсэг нь нийт ажлын ердөө 2–3% гэсэн үг: $\frac{1}{S+(1-S)/2} = 1.95 \Rightarrow S \approx 0.026$.

Гурван талт хуваалтын ач тус тодорхой. `FEW_UNIQUE` дээр Quicksort ердөө **5** зангилаа, `ALL_EQUAL` дээр **1** зангилаа үүсгэв. Mergesort эсрэгээрээ өгөгдлийг огт "хардаггүй" тул үргэлж 16 383 зангилаа үүсгэнэ. Энэ нь хоёр алгоритмын гүн ялгааг харуулна: Quicksort өгөгдлийн *бүтцэд дасан зохицдог*, Mergesort *сохор* ч баталгаатай.

Mergesort эрэмбэлэгдсэн өгөгдөл дээр 22 дахин хурдан. Үүний шалтгаан нь `merge` доторх `if (a[mid-1] <= a[mid]) return;` гэсэн нэг мөр — хоёр хэсэг аль хэдийн зөв дараалалтай бол хуулалт хийхгүй. Энэ бол Timsort-ын үндсэн санааны хамгийн энгийн хэлбэр.

Дүрслэл нь алдаа илрүүлэх хэрэгсэл болсон. Pivot сонголтын алдааг *хугацааны тоо биш, модны хэлбэр* илрүүлсэн: эрэмбэлэгдсэн өгөгдөл дээр мод нэг тал руугаа урт гинж үүсгэж байгаа нь шууд харагдсан. Энэ нь "UI дүрслэл нь зүгээр чимэглэл биш" гэсэн даалгаврын шаардлагын бодит баталгаа юм.

9.3 Хязгаарлалт

1. **Хоёр цөмт орчин.** Хурдасгалтын дээд хязгаар 2 байв. Олон цөмт машин дээр H3-ыг илүү өргөн хүрээнд шалгах боломжтой.
2. `Integer` **объектын массив.** Сурах бичиг `Comparable` шаарддаг тул анхдагч `int[]` ашиглах боломжгүй. Boxing нь кэшийн ашиглалтыг муутгаж, абсолют хугацааг 2–3 дахин удаашруулдаг. Харьцуулалт бүгд ижил нөхцөлд хийгдсэн тул *дүгнэлтэд нөлөөлөхгүй*.
3. **Бүртгэл нь хэмжилтийг гажуудуулна.** UI-д бүртгэл үргэлж асаалттай тул тэнд харагдах хугацааг цэвэр хэмжилт гэж үзэж болохгүй. Тайлангийн бүх тоо `-no-trace` горимоор авагдсан.

4. **Тогтвортой байдал (stability).** Хоёр алгоритм хоёулаа *тогтворгүй* (ижил утгатай элементүүдийн анхны дараалал хадгалагдахгүй). Mergesort-ийг тогтвортой болгох боломжтой ч insertion sort-ийн навч ба `merge`-ийн харьцуулалтыг нарийн тохируулах шаардлагатай.
5. **Том модны дүрслэл.** 200 000 зангилаатай мод зурахад удаана. Дүрслэлийг $n \leq 5000$ хүрээнд ашиглахыг зөвлөнө.

9.4 Цаашид хийж болох ажил

Параллель merge (нэгтгэх алхмыг ч параллельчлах) нь Amdahl-ын цуваа хэсгийг багасгана. Мөн `ForkJoinTask.getSurplusQueuedTaskCount()` ашиглан threshold-ийг *динамикаар* тохируулах нь орчин үеийн сангуудын хандлага юм.

10 Дүгнэлт

10.1 Хэрэгжсэн зүйл

1. Сурах бичгийн Programming Project 3-ын **бүх** шаардлагыг хангасан: fork-join API, `RecursiveAction`, Quicksort ба Mergesort хоёр хувилбар, threshold-ооc доош insertion sort, `Comparable` ерөнхий төрөл.
2. Багшийн шаардсан **рекурсийн модны** интерактив дүрслэл: fork-join задрал, дэд ажлууд, join үе шат, ажилчин урсгалын өнгө, алхамчилсан горим, зум ба зангилааны дэлгэрэнгүй.
3. Оролт, threshold, урсгалын тоо, өгөгдлийн хэв маяг, seed бүгд өөрчлөгдөж дахин туршигдана. Нэг ч утга эх кодод хатуу тогтоогдоогүй.
4. Хүчингүй оролтыг барьж, монгол хэлээр ойлгомжтой мэдээлнэ.
5. Өөр компьютерт Manual-ийн дагуу дахин ажиллуулах боломжтой: гадны сангүй, build хэрэгсэлгүй, дэлгэцгүй горимтой.

10.2 Нотолсон үр дүн

1. **58/58** тест давсан; зөв ажиллагааг `Arrays.sort`-той тулган баталсан.
2. Хоёр цөм дээр **1.95–2.15** дахин хурдасгалт — онолын дээд хязгаарт хүрсэн.
3. Threshold-ийн оновчтой муж **50–500** гэдгийг хэмжилтээр тогтоосон бөгөөд энэ нь сурах бичгийн зөвлөмжтэй нийцэв.
4. Гурван талт хуваалт нь давхардсан утгатай өгөгдөл дээр модыг 16 383 зангилаанаас **1** болгон багасгасныг тоогоор харуулав.
5. Pivot сонголтын далд алдааг *дүрслэлээр* илрүүлж, засаад эрэмбэлэгдсэн өгөгдөл дээрх хугацааг **2310 мс → 196 мс** болгосон.

10.3 Сурсан зүйл

Fork-join параллелизмд хамгийн чухал шийдвэр нь алгоритм сонголт биш, *хэзээ хуваахаа зогсоох* (threshold) ба *хэрхэн хуваах* (pivot) хоёр юм. Эдгээр хоёр параметр нь ижил алгоритмын гүйцэтгэлийг хэдэн арван дахин өөрчилж чадна. Мөн параллель программын алдааг *хугацааны тоо* бус *бүтцийн дүрслэл* илүү хурдан илрүүлдэг нь энэ ажлын хамгийн практик сургамж байлаа.

Ашигласан материал

[1] Silberschatz, A., Galvin, P. B., Gagne, G. *Operating System Concepts*, 10-р хэвлэл. Wiley, 2018. Бүлэг 4 — Threads & Concurrency: §4.5.2 Fork Join (х. 180–183), Figure 4.18 `SumTask` (х. 183),

Figure 4.19 UML (x. 184); Programming Projects — Project 3 Fork-Join Sorting Application (x. P-27–P-28), Project 2 Multithreaded Sorting Application (x. P-26–P-27, Figure 4.27).

[2] Oracle. *Java Platform SE 21 API Specification* — `java.util.concurrent.ForkJoinPool`, `RecursiveAction`.

[3] Lea, D. *A Java Fork/Join Framework*. ACM Java Grande, 2000.

[4] Dijkstra, E. W. *A Discipline of Programming*. Prentice-Hall, 1976 — Dutch national flag problem.

[5] Bentley, J. L., McIlroy, M. D. *Engineering a Sort Function*. Software: Practice and Experience, 1993.

Бэлэн код, сан, AI ашигласан тухай. Гадны сан, бэлэн эрэмбэлэх код ашиглаагүй — бүх алгоритм энэ репозиторийн дотор бичигдсэн. AI туслах (Claude) ашиглан кодыг хянах, алдаа оношлох, тайлан бэлтгэхэд дэмжлэг авсан. Алгоритмын бүх шийдэл, хэмжилтийн бүх тоо нь энэ репозиторийн кодыг бодитоор ажиллуулж гаргасан үр дүн бөгөөд `./test.sh`, `./run.sh -bench` командуудаар дахин давтагдана.